

Middleware Architecture (IS3108/SCS3203)

Scenario-Based Class Assessment – 2023

Assignment Scenario: SriBank PLC

Answer for Exercise 1:

1a. Layers and Tiers of the Architecture:

The proposed architecture follows an N-tier architecture:

- **Presentation Layer (Client Tier)**
- **Business Logic Layer (Application Tier)**
- **Integration Layer (Middleware Tier)**
- **Data Access Layer (Database Tier)**
- **External Systems Layer**

1b. Components of Each Layer/Tier:

• **Presentation Layer:**

- Web Browser (Internet Banking)
- Mobile App (Android/iOS)
- Responsive UI (HTML5, CSS3, JavaScript/React/Angular)

• **Business Logic Layer:**

- Authentication and Authorization Module (2FA)
- Session Management
- Core Services (Balance Inquiry, Fund Transfer, Bill Payment, etc.)
- Workflow Services (Loan Requests, Account Creation, Credit Limit Requests)

• **Integration Layer:**

- API Gateway
- Service Adapters (REST, SOAP, ISO 8583, POX)
- Enterprise Service Bus (ESB)

- **Data Access Layer:**

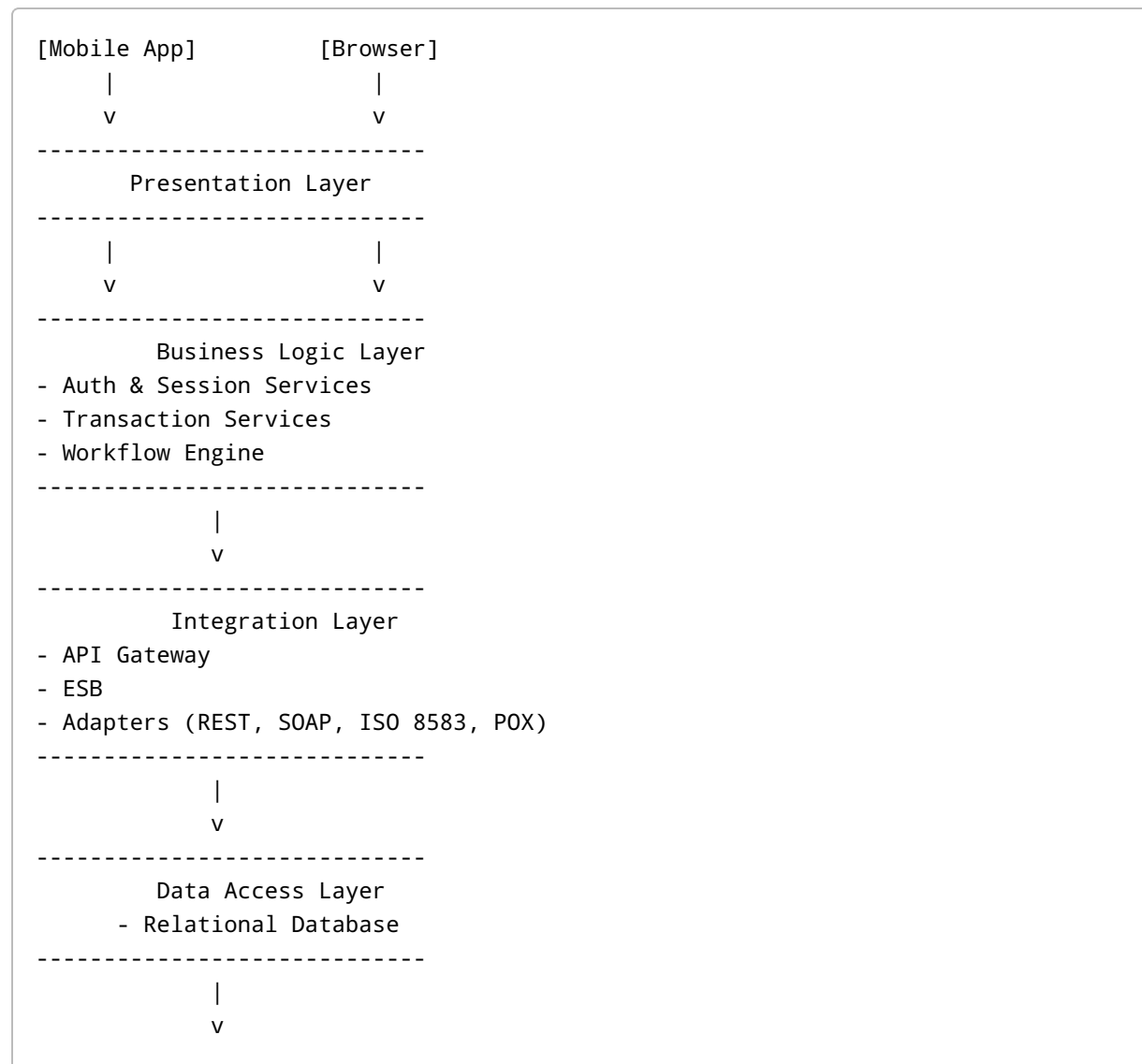
- Relational Database (e.g., PostgreSQL, MySQL)

- **External Systems Layer:**

- Core Banking System (REST)
- Inter-bank Payment Gateway (ISO 8583)
- Credit Card System (SOAP)
- Internal Payment Switch (POX over TCP/IP)

1c. Sketch of the Overall Architecture:

(Drawing to be submitted by student on paper. Description below helps sketch manually)



```
-----  
    External Systems Layer  
- Core Banking (REST)  
- Interbank Gateway (ISO 8583)  
- Credit Card System (SOAP)  
- Payment Switch (POX)  
-----
```

1d. Architectural Patterns Used:

1. **N-Tier Architecture** - Provides clear separation of concerns between presentation, business logic, and data access.
2. **Service-Oriented Architecture (SOA)** - Supports integration with external heterogeneous systems via service contracts.
3. **Microkernel Pattern** - Used to support workflow-based features that require plug-in functionality (e.g., credit limit request).

Answer for Exercise 2:

Recommended Pattern: Model-View-Controller (MVC)

- **Why MVC?**

- Enables separation of UI logic from business logic.
- Facilitates client-side validation (via JavaScript/React/Angular).
- Enhances user responsiveness and experience.
- Supports modular and testable UI components.

- **Client Responsibilities:**

- Input validation
- UI rendering and navigation
- Consuming REST APIs for dynamic data

Answer for Exercise 3:

Most Suitable Architectural Pattern:

Enterprise Service Bus (ESB)

- **Why ESB?**

- Allows orchestration of services.
- Supports multiple communication protocols.
- Decouples systems and enables centralized management of integration.

3 Suitable Integration Patterns within ESB:

1. Adapter Pattern:

2. Converts different protocols (REST, SOAP, ISO 8583, POX) to a unified format.

3. Message Routing Pattern:

4. Dynamically routes messages based on content/type of request (e.g., route credit card requests to SOAP endpoint).

5. Message Transformation Pattern:

6. Transforms data formats to be compatible between the banking platform and external systems (e.g., XML to JSON).

Answer for Exercise 4:

Nature of Communication for Notifications:

- **Asynchronous Communication**
- Push notifications and SMS are event-driven and non-blocking.

Recommended Pattern:

Publisher-Subscriber Pattern (Pub/Sub)

- **Why Pub/Sub?**
- Efficient for sending notifications to many subscribers.
- Decouples the producer (banking platform) from the consumers (mobile apps).
- Supports scalable and real-time event distribution.
- **Technologies:**
- Firebase Cloud Messaging (for push)
- SMS Gateway API (for SMS)

Answer for Exercise 5:

Most Suitable Layer for Managing Transaction Concurrency:

Business Logic Layer (Application Tier)

- **Why?**
- Central point for enforcing business rules and managing data integrity.
- Can leverage transaction managers to maintain ACID properties.
- Ensures operations like fund transfers are atomic and isolated.

Concurrency Management Techniques:

- Pessimistic Locking (when high contention is expected)
- Optimistic Locking (when low contention)
- Database Transaction Isolation Levels (e.g., Serializable, Repeatable Read)

Assumptions:

- Retail customers only.
- Real-time transactions require immediate feedback.
- Workflow services are asynchronous and require human/system intervention.
- Mobile and web clients share the same backend services.

End of Assignment Answers